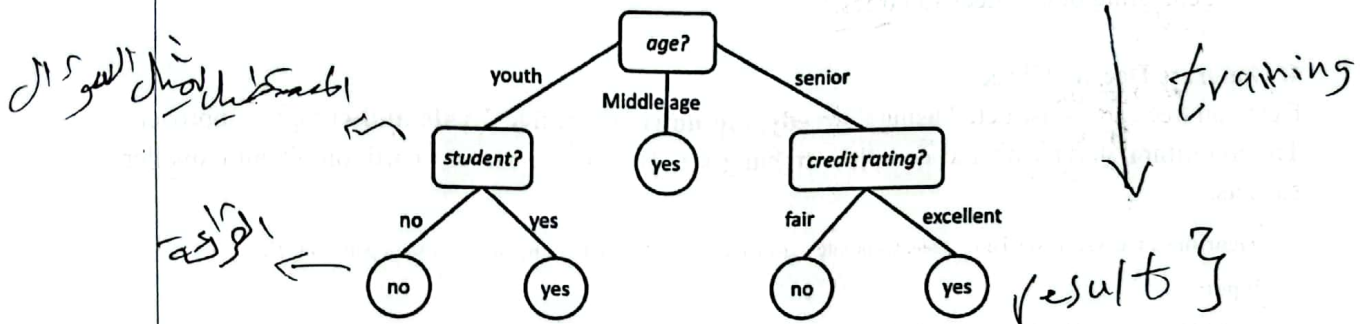


Decision Trees (شجرة القرار)

1. What is a Decision Tree (DT)?

Decision tree is the learning of decision trees from class-labeled training tuples. It is a flowchart-like tree structure used for classification, where each.

- **Internal Node (Nonleaf Node):** Denotes a test on an attribute (feature). (Represented by rectangles in a typical figure, such as the *age* or *student* tests).
- **Branch:** Represents an outcome of the test at the internal node.
- **Leaf Node (Terminal Node):** Holds a class label (the prediction). (Represented by ovals/circles, labeled 'yes' or 'no').
- **Root Node:** The topmost node in the tree.



A decision tree for the concept *buys_computer*, indicating whether a customer is likely to purchase a computer. Each internal (nonleaf) node represents a test on an attribute. Each leaf node represents a class (either *buys_computer* = yes or *buys_computer* = no).

Classification with a Decision Tree:

To classify a data tuple with an unknown class label, its attribute values are tested against the tree, following a path from the root node to a leaf node, which gives the class prediction.

2. Why are Decision Tree Classifiers Popular?

Decision trees are widely used because they offer several key advantages:

1. Easy to explain (even for non-expert),
2. Easy to implement (many software)
3. Efficient
4. Can tolerant missing data
5. *White box
6. No need to normalize data
7. Non-parametric: No assumption on data distribution, no assumption on attribute independency
8. Can work on various attribute types

Disadvantages of Decision Tree

1. Unstable. Sensitive to noise
2. Accuracy may be not good enough (depending on your data)
3. The optimal splitting is NP. Greedy algorithms are used
4. Overfitting

3. Decision Tree Induction

Decision Tree Induction is a cornerstone technique in supervised learning, developed through seminal work in the late 1970s and 1980s.

Key Algorithms and Developers

- **ID3 (Iterative Dichotomizer):** Developed by J. Ross Quinlan in the late 1970s and early 1980s. It built upon earlier work in concept learning by E. B. Hunt, J. Marin, and P. T. Stone.
- **C4.5:** Quinlan's successor to ID3, which became a standard benchmark for comparing new supervised learning algorithms.
- **CART (Classification and Regression Trees):** Developed independently around 1984 by a group of statisticians (L. Breiman, J. Friedman, R. Olshen, and C. Stone). CART focuses on generating binary decision trees.

4. Generate Decision Tree

Decision trees are constructed using a **greedy, top-down, recursive divide-and-conquer** approach. The algorithm starts with the complete training set, D , and recursively partitions it into smaller subsets.

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition, D .

Input:

- Data partition, D , which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split-point* or *splitting_subset*.

Output: A decision tree.

Method:

- (1) create a node N ;
- (2) if tuples in D are all of the same class, C , then
- (3) return N as a leaf node labeled with the class C ;
- (4) if *attribute_list* is empty then
- (5) return N as a leaf node labeled with the majority class in D ; // majority voting
- (6) apply *Attribute_selection_method*(D , *attribute_list*) to find the "best" *splitting_criterion*;
- (7) label node N with *splitting_criterion*;
- (8) if *splitting_attribute* is discrete-valued and
multiway splits allowed then // not restricted to binary trees
- (9) *attribute_list* $\leftarrow$ *attribute_list* - *splitting_attribute*; // remove *splitting_attribute*
- (10) for each outcome j of *splitting_criterion*
// partition the tuples and grow subtrees for each partition
- (11) let D_j be the set of data tuples in D satisfying outcome j ; // a partition
- (12) if D_j is empty then
- (13) attach a leaf labeled with the majority class in D to node N ;
- (14) else attach the node returned by *Generate_decision_tree*(D_j , *attribute_list*) to node N ;
- endfor
- (15) return N .

5. Attribute Selection Measures

These are heuristics used to select the **splitting criterion** that "best" partitions the data into individual classes. The goal is to make the resulting partitions as "pure" as possible (where all tuples belong to the same class).

A. Information gain (Entropy)

ID3 uses **information gain** as its attribute selection measure. Let node N represent or hold the tuples of partition D . The attribute with the highest information gain is chosen as the splitting attribute for node N .

- Let p_i be the probability that an arbitrary tuple in D belongs to class C_i , estimated by $|C_{i,D}|/|D|$
- Expected information (entropy) needed to classify a tuple in D :

$$Info(D) = -\sum_{i=1}^m p_i \log_2(p_i)$$

Information gain

- Information needed (after using A to split D into v partitions) to classify D :

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

- Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

Example:-

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

To find the splitting criterion for these tuples,

1. Compute the expected information needed to classify a tuple in D :

Information data $\leftarrow Info(D) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940$ bits.

2. Next, we need to compute the expected information requirement for each attribute. Let's start with the attribute age .

$$Info_{age}(D) = \frac{5}{14} \times \left(-\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right) \right) + \frac{4}{14} \times \left(-\frac{4}{4} \log_2\left(\frac{4}{4}\right) \right) + \frac{5}{14} \times \left(-\frac{3}{5} \log_2\left(\frac{3}{5}\right) - \frac{2}{5} \log_2\left(\frac{2}{5}\right) \right)$$

$$= 0.694 \text{ bits.} \rightarrow Info_{age}$$

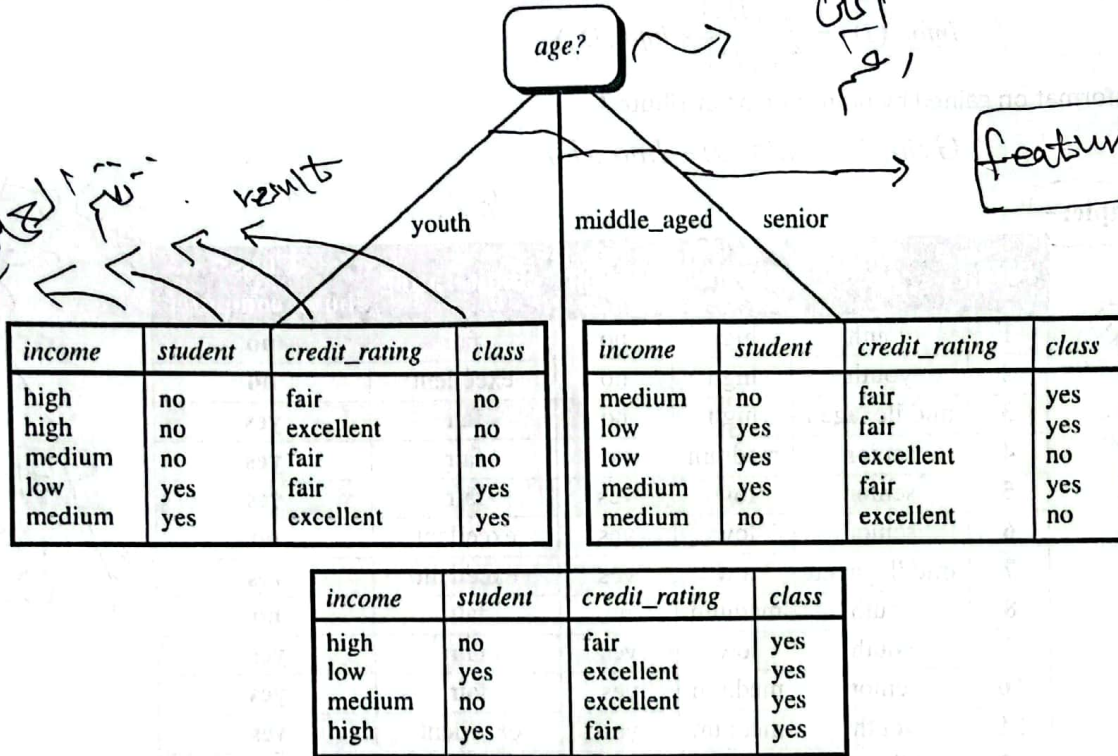
Hence, the gain in information from such partitioning would be

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

Similarly, we can compute

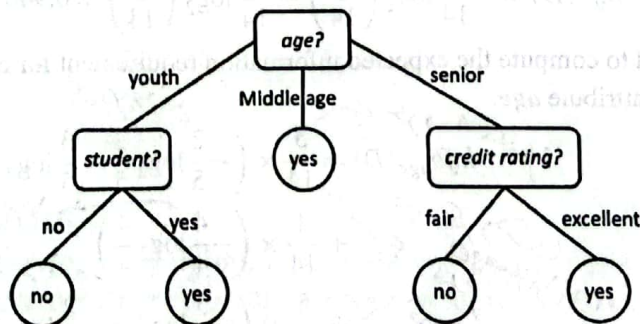
$$\begin{aligned} \text{Gain}(\text{income}) &= 0.029 \text{ bits,} \\ \text{Gain}(\text{student}) &= 0.151 \text{ bits, and} \\ \text{Gain}(\text{credit_rating}) &= 0.048 \text{ bits.} \end{aligned}$$

Because **age** has the highest information gain among the attributes, it is selected as the splitting attribute.



The attribute **age** has the highest information gain and therefore becomes the splitting attribute at the root node of the decision tree. Branches are grown for each outcome of **age**. The tuples are shown partitioned accordingly.

3. After choosing the age, we got three tables (subsets). We repeat the process of selecting the best tuple for splitting and continue.
4. The result final.



B. Gain ratio

Information gain measure is biased towards attributes with a large number of values (e.g. ID)

Solution (Gain Ratio):

Handwritten notes: "يتميز لا كبر عدد من الصفات" (It is characterized by a large number of attributes), "مثلا عدد" (For example, number), "ID الطلاب إذا كان كبير" (Students' ID if it is large).

C4.5, a successor of ID3, uses an extension to information gain known as **gain ratio**, which attempts to overcome this bias. It applies a kind of normalization to information gain using a "split information" value defined analogously with $Info(D)$ as

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \times \log_2 \left(\frac{|D_j|}{|D|} \right).$$

The gain ratio is defined as

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}.$$

Example Computation of gain ratio for the attribute *income*.

A test on *income* splits the data of above table into three partitions, namely *low*, *medium*, and *high*, containing four, six, and four tuples, respectively. To compute the gain ratio of *income*, we first obtain.

$$SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) - \frac{6}{14} \times \log_2 \left(\frac{6}{14} \right) - \frac{4}{14} \times \log_2 \left(\frac{4}{14} \right) = 1.557.$$

we have $Gain(income) = 0.029$. Therefore $GainRatio(income) = \frac{0.029}{1.557} = 0.019$

C. Gini

The **Gini** is used in **CART**. The Gini measures the impurity of D , the Gini impurity considers a **binary split** for each attribute.

For example, if *income* has three possible values, namely {*low*, *medium*, *high*}, then the possible subsets are {*low*, *medium*}, {*low*, *high*}, {*medium*, *high*}, {*low*}, {*medium*}, {*high*}.

Example Induction of a decision tree using the Gini.

Let D be the training data shown earlier, where there are nine tuples belonging to the class *buys_computer* = *yes* and the remaining five tuples belong to the class *buys_computer* = *no*. A (root) node N is created for the tuples in D . We first compute the Gini of D :

$$Gini(D) = 1 - \left(\frac{9}{14} \right)^2 - \left(\frac{5}{14} \right)^2 = 0.459.$$

To find the splitting criterion for the tuples in D , we need to compute the Gini impurity for each attribute. Let's start with the attribute *income* and consider each of the possible splitting subsets. Consider the subset {*low*, *medium*}. This would result in 10 tuples in partition D_1 satisfying the condition "*income* $\in$ {*low*, *medium*}." The remaining four tuples of D would be assigned to partition D_2 . The Gini value computed based on this partitioning is

$$\begin{aligned} Gini_{income \in \{low, medium\}}(D) &= \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2) \\ &= \frac{10}{14} \left(1 - \left(\frac{7}{10} \right)^2 - \left(\frac{3}{10} \right)^2 \right) + \frac{4}{14} \left(1 - \left(\frac{2}{4} \right)^2 - \left(\frac{2}{4} \right)^2 \right) \\ &= 0.443 \\ &= Gini_{income \in \{high\}}(D). \end{aligned}$$

Similarly, the Gini values for splits on the remaining subsets are 0.458 (for the subsets {*low*, *high*} and {*medium*}) and 0.450 (for the subsets {*medium*, *high*} and {*low*}). Therefore the best binary split for attribute *income* is on {*low*, *medium*} (or {*high*}) because it **minimizes** the Gini impurity.


```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'age': ['y', 'y', 'm', 's', 's', 's', 'm', 'y', 'y', 's', 'y', 'm', 'm', 's'],
    'income': ['h', 'h', 'h', 'm', 'l', 'l', 'l', 'm', 'l', 'm', 'm', 'm', 'h', 'm'],
    'student': ['n', 'n', 'n', 'n', 'y', 'y', 'y', 'n', 'y', 'y', 'y', 'n', 'y', 'n'],
    'credit': ['f', 'e', 'f', 'f', 'f', 'e', 'e', 'f', 'f', 'f', 'e', 'e', 'f', 'e'],
    'Class': [0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0]
}
df = pd.DataFrame(data)
# ترميز الخصائص النصية إلى أرقام
le = LabelEncoder()
for col in ['age', 'income', 'student', 'credit']:
    df[col] = le.fit_transform(df[col])
X = df[['age', 'income', 'student', 'credit']]
y = df['Class']
# تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
# إنشاء النموذج وتدريبه
tree = DecisionTreeClassifier(criterion='entropy', random_state=42)
tree.fit(X_train, y_train)
# عرض الدقة
print("Accuracy on training set: {:.3f}".format(tree.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(tree.score(X_test, y_test)))
# رسم شجرة القرار داخل البرنامج
plt.figure(figsize=(12, 8))
plot_tree(tree,
           feature_names=['age', 'income', 'student', 'credit'],
           class_names=['no', 'yes'],
           filled=True, # بلون العنود
           rounded=True, # زوايا دائرية
           fontsize=10)
plt.title("Decision Tree for buys_computer dataset")
plt.show()

```

Information Gain (ID3)

شرح هذا البرنامج خطوة بخطوة مع نتائج كل خطوة، بشكل واضح ومبسط. هذا البرنامج يقوم بإنشاء شجرة قرار (Decision Tree) لتصنيف الأشخاص حسب ما إذا كانوا سيشترون جهاز كمبيوتر أم لا بناءً على بعض الخصائص.

1 استيراد المكتبات

```

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import pandas as pd
import matplotlib.pyplot as plt

```

شرح:

- DecisionTreeClassifier : لبناء شجرة القرار.
- plot_tree : لرسم الشجرة.
- train_test_split : لتقسيم البيانات إلى مجموعة تدريب واختبار.
- LabelEncoder : لتحويل البيانات النصية (مثل 'y', 'h', 'n') إلى أرقام.
- pandas : لإدارة البيانات في شكل جدول (DataFrame).
- matplotlib.pyplot : لرسم الشجرة.

2 إنشاء البيانات

```

data = {
    'age': ['y', 'y', 'm', 's', 's', 's', 'm', 'y', 'y', 's', 'y', 'm', 'm', 's'],
    'income': ['h', 'h', 'h', 'm', 'l', 'l', 'l', 'm', 'l', 'm', 'm', 'm', 'h', 'm'],

```

```

'student': ['n','n','n','n','y','y','y','n','y','y','y','n','y','n'],
'credit': ['f','e','f','f','f','e','e','f','f','f','e','e','f','e'],
'Class': [0,0,1,1,1,0,1,0,1,1,1,1,1,0]
}
df = pd.DataFrame(data)

```

شرح:

- البيانات تحتوي على 4 خصائص (age, income, student, credit) وهدف (Class).
- Class = 1 يعني سيشتري الكمبيوتر، 0 يعني لن يشتريه.

3 ترميز البيانات النصية إلى أرقام

```

le = LabelEncoder()
for col in ['age', 'income', 'student', 'credit']:
    df[col] = le.fit_transform(df[col])

```

شرح:

- يحول النص إلى أرقام LabelEncoder.

مثال:

- age ['y','m','s'] → [2,1,0]
- income ['h','m','l'] → [2,1,0]
- student ['y','n'] → [1,0]
- credit ['f','e'] → [1,0]

بعد الترميز، جدول البيانات يصبح رقمي بالكامل.

4 تجهيز خصائص الهدف والمتغيرات

```

X = df[['age', 'income', 'student', 'credit']]
y = df['Class']

```

- X : كل الخصائص (features).
- y : الهدف (target).

5 تقسيم البيانات إلى تدريب واختبار

```

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

```

- تدريب، 25% اختبار default = 75% هنا نسبة) يقوم بتقسيم البيانات بشكل عشوائي train_test_split.
- لجعل التقسيم متكرر بنفس الطريقة دائماً random_state=42.

مثال على حجم البيانات بعد التقسيم:

- تدريب: 10 صفوف
- اختبار: 4 صفوف

6 إنشاء نموذج شجرة القرار وتدريبه

```

tree = DecisionTreeClassifier(criterion='entropy', random_state=42)
tree.fit(X_train, y_train)

```

- لنقسم العقد Entropy / Information Gain يعني أن الشجرة تستخدم مقياس criterion='entropy'.
- تدريب النموذج على بيانات التدريب : tree.fit

إذا لم تستطع فهم الشجرة أو اختبارها، فراجع الفيديو التالي:

7 تقييم النموذج

```

print("Accuracy on training set: {:.3f}".format(tree.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(tree.score(X_test, y_test)))

```

- نسبة التوقعات الصحيحة (accuracy) بحسب الدقة tree.score.

مثال نتائج ممكنة:

Accuracy on training set: 1.000

Accuracy on test set: 0.750

- التدريب كامل: 100% (شجرة القرار تتذكر البيانات).
- اختبار: 75% (قد تفشل في 1 من 4 حالات).

8 رسم شجرة القرار plt.figure

Example

```
# استيراد المكتبات الضرورية
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

# تحميل بيانات سرطان الثدي
cancer = load_breast_cancer()

# تقسيم البيانات إلى تدريب واختبار
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data,
    cancer.target,
    stratify=cancer.target, # للحفاظ على نسبة الفئات
    random_state=42
)

# إنشاء نموذج شجرة القرار باستخدام مقياس entropy
tree = DecisionTreeClassifier(criterion='entropy', random_state=0)

# تدريب النموذج
tree.fit(X_train, y_train)

# عرض الدقة على بيانات التدريب والاختبار
print("Accuracy on training set: {:.3f}".format(tree.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(tree.score(X_test, y_test)))
```

Knn/classifier :-	Decision Tree
1- classifier (تصنيف) في كل مرة تراجع الى البيانات لكي تظهر نتيجة.	1- Making Pattern في كل قرار تراجع الى النمط والقاعدة ال (label) الذي تصاحبه للتصنيف (0, 1).
2- كلاهما من نوعا للتصنيف 3- ابطيء	2- يستخدم نوعا للتصنيف 3- أسرع